EHM(X): 形式規則と主要定理

System F^{ξ} に集合効果 handler と HM(X) の制約体系を加える rank-1 表層

図 1. 構文と意味論

$$
\begin{aligned}
&e ::= x \mid n \mid \mathbf{true} \mid \mathbf{false} \mid \lambda x.e \mid e_1\, e_2 \mid \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 \mid \mathbf{perform}\ op\ e \mid \mathbf{handle}\ e\ \mathbf{with}\ h,\\
&h ::= \{\mathbf{return}\ r \mapsto e_r;\ op(x;k) \mapsto e_o\}, \qquad v ::= x \mid n \mid \mathbf{true} \mid \mathbf{false} \mid \lambda x.e,\\
&\tau ::= \alpha \mid \iota \mid \tau \xrightarrow{\epsilon} \tau, \qquad \epsilon ::= \varnothing \mid \{op\} \mid \mu \mid \epsilon \cup \epsilon,\\
&\sigma ::= \forall \bar{\alpha}, \bar{\mu}.\ C \Rightarrow \tau,\\
&\sigma_c ::= \forall \bar{\alpha}, \bar{\mu}.\ C \Rightarrow (\tau ! \epsilon), \qquad C \in \mathcal{C}_X.
\end{aligned}
$$

ι は閉じた基本型を表し，$Int, Bool$ を含む．$\mathcal{C}_X$ は制約体系 X の制約全体であり，型等式・効果制約・クラス制約を同じ集合に含む．

$$\epsilon \equiv \epsilon' \quad \overset{\text{def}}{\Longleftrightarrow} \quad (\epsilon \subseteq \epsilon') \wedge (\epsilon' \subseteq \epsilon).$$

$$I_\rho(\varnothing) = \varnothing, \quad I_\rho(\{op\}) = \{op\}, \quad I_\rho(\mu) = \rho_E(\mu), \quad I_\rho(\epsilon_1 \cup \epsilon_2) = I_\rho(\epsilon_1) \cup I_\rho(\epsilon_2).$$

$$I_\rho(\alpha) = \rho_T(\alpha), \quad I_\rho(\iota) = \iota, \quad I_\rho(a \xrightarrow{\epsilon} b) = I_\rho(a) \xrightarrow{I_\rho(\epsilon)} I_\rho(b).$$

$$\boxed{\mathrm{Split}(op, \epsilon, \delta) \overset{\text{def}}{=} (\delta \subseteq \epsilon) \wedge (\epsilon \subseteq \{op\} \cup \delta) \wedge (op \notin \delta)}$$

図 2. 値呼びの簡約規則

$$(\lambda x.e)\, v \longrightarrow e[x \mapsto v] \qquad \mathbf{handle}\ v\ \mathbf{with}\ h \longrightarrow e_r[r \mapsto v].$$

$$\mathbf{handle}\ E[\mathbf{perform}\ op\ v]\ \mathbf{with}\ h \longrightarrow e_o[x \mapsto v][k \mapsto k_E], \quad op \notin \mathrm{handled}(E), \qquad k_E = \lambda y.\mathbf{handle}\ E[y]\ \mathbf{with}\ h.$$

宣言的型付け　$C; \Gamma \vdash e : \tau \mid \epsilon$

図 3. 基本的な型付け規則

$$\frac{\Gamma(x) = \sigma = \forall \bar{\beta}.C_x \Rightarrow \tau \quad \theta\beta \notin B\ (\beta \in B), \quad \theta\gamma = \gamma\ (\gamma \notin B) \quad C \models_X \mathrm{Inst}_X(\sigma, \theta)}{C; \Gamma \vdash x : \theta\tau \mid \varnothing}\ (\textsc{Var})$$

$$\frac{C; \Gamma, x : a \vdash e : b \mid \eta}{C; \Gamma \vdash \lambda x.e : a \xrightarrow{\eta} b \mid \varnothing}\ (\textsc{Abs})$$

$$\frac{C; \Gamma \vdash e_1 : a \xrightarrow{\eta} b \mid \epsilon_1 \quad C; \Gamma \vdash e_2 : a \mid \epsilon_2}{C; \Gamma \vdash e_1\, e_2 : b \mid \epsilon_1 \cup \epsilon_2 \cup \eta}\ (\textsc{App})$$

$$\frac{\Sigma(op) = \forall \bar{\beta}.\tau_a \to \tau_b \quad C; \Gamma \vdash e : \tau_a[\bar{\theta}/\bar{\beta}] \mid \eta}{C; \Gamma \vdash \mathbf{perform}\ op\ e : \tau_b[\bar{\theta}/\bar{\beta}] \mid \eta \cup \{op\}}\ (\textsc{Perform})$$

$$\frac{n \in \mathbb{Z}}{C; \Gamma \vdash n : Int \mid \varnothing}\ (\textsc{Int}) \qquad \frac{b \in \{\mathbf{true}, \mathbf{false}\}}{C; \Gamma \vdash b : Bool \mid \varnothing}\ (\textsc{Bool})$$

$$\boxed{\begin{aligned}&\mathrm{Inst}_X(\forall B.P \Rightarrow \tau, \theta) = \exists B.\big(P \wedge E_\theta(B)\big),\\ &E_\theta(B) = \bigwedge_{\alpha \in B_T} (\alpha = \theta\alpha) \wedge \bigwedge_{\mu \in B_E} (\mu \equiv \theta\mu).\end{aligned}}$$

図 4. 値制限付き let

$$\frac{C_1;\Gamma \vdash v : \tau_1 \mid \varnothing \quad A = \mathrm{FV}(C_1,\tau_1) \setminus \mathrm{FV}(\Gamma) \quad \sigma = \forall A.C_1 \Rightarrow \tau_1, \quad R_1 = \exists A.C_1, \quad C_2;\Gamma, x:\sigma \vdash e_2 : \tau_2 \mid \epsilon_2}{R_1 \wedge C_2;\Gamma \vdash \mathbf{let}\ x = v\ \mathbf{in}\ e_2 : \tau_2 \mid \epsilon_2} \quad (\textsc{LetVal})$$

$$\frac{\neg\,\mathrm{Value}(e_1) \quad C;\Gamma \vdash e_1 : \tau_1 \mid \epsilon_1 \quad C;\Gamma, x:\tau_1 \vdash e_2 : \tau_2 \mid \epsilon_2}{C;\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2 \mid \epsilon_1 \cup \epsilon_2} \quad (\textsc{LetMono})$$

図 5. 深い handler の型付け

$$\frac{\begin{array}{c}\Sigma(op) = \forall\overline{\beta :: \kappa}.\tau_a \to \tau_b \qquad C;\Gamma \vdash e : \tau_{in} \mid \epsilon_{in} \\ C;\Gamma, r:\tau_{in} \vdash e_r : \tau_{out} \mid \epsilon_r \\ D;\Gamma, x:\tau_a, k:\tau_b \xrightarrow{\chi} \tau_{out} \vdash e_o : \tau_o \mid \epsilon_o \\ C \models_X \exists U_T, U_E.(D \wedge \tau_o = \tau_{out} \wedge \epsilon_o \subseteq \chi) \quad (\bar\beta \text{ は剛性名}) \\ C \models \mathrm{Split}(op, \epsilon_{in}, \delta) \quad C \models \delta \subseteq \chi \quad C \models \epsilon_r \subseteq \chi \\ \mathrm{ClauseScope}(C, \Gamma, \tau_{in}, \tau_{out}, \epsilon_{in}, \epsilon_r, \delta, \chi, U_T, U_E)\end{array}}{C;\Gamma \vdash \mathbf{handle}\ e\ \mathbf{with}\ \{\mathbf{return}\ r \mapsto e_r;\ \ op(x;k) \mapsto e_o\} : \tau_{out} \mid \chi} \quad (\textsc{Handle})$$

$\bar\beta$ は操作節で全称的に検査する，　$\bar\beta \cap \mathrm{FV}(\Gamma, \tau_{in}, \tau_{out}, \epsilon_{in}, \epsilon_r, \chi, \delta) = \varnothing.$

$$\begin{array}{c} U_T \cap \mathrm{FV}_T(\Gamma, C, \tau_{in}, \tau_{out}, \tau_a, \tau_b, \bar\beta) = \varnothing, \\ U_E \cap \mathrm{FV}_E(\Gamma, C, \tau_{in}, \tau_{out}, \epsilon_{in}, \epsilon_r, \delta, \chi, \tau_a, \tau_b, \bar\beta) = \varnothing, \\ \bar\beta \cap \mathrm{FV}(C) = \varnothing. \end{array}$$

操作宣言の束縛名は，必要なら捕獲を避けて α 改名する．

制約生成　$\mathrm{Gen}_\Gamma(e) = (\tau, \epsilon, C)$

図 6. 基本的な制約生成

$$\mathrm{Gen}_\Gamma(n) = (\mathit{Int}, \varnothing, \top), \qquad \mathrm{Gen}_\Gamma(b) = (\mathit{Bool}, \varnothing, \top) \quad (b \in \{\mathbf{true}, \mathbf{false}\}).$$

$$\Gamma(x) = \sigma = \forall B.C_x \Rightarrow \tau_x, \quad \theta B \text{ は新しい名前} \Longrightarrow \mathrm{Gen}_\Gamma(x) = (\theta\tau_x, \varnothing, \mathrm{Inst}_X(\sigma, \theta)),$$

$$\mathrm{Gen}_{\Gamma, x:a}(e) = (b, \eta, C) \Longrightarrow \mathrm{Gen}_\Gamma(\lambda x.e) = (a \xrightarrow{\eta} b, \varnothing, C).$$

$$\mathrm{Gen}_\Gamma(e_1) = (\tau_1, \epsilon_1, C_1), \quad \mathrm{Gen}_\Gamma(e_2) = (\tau_2, \epsilon_2, C_2) \Longrightarrow \mathrm{Gen}_\Gamma(e_1\, e_2) = (b, \epsilon_1 \cup \epsilon_2 \cup \eta, C_1 \wedge C_2 \wedge (\tau_1 = \tau_2 \xrightarrow{\eta} b)).$$

$$\Sigma(op) = \forall\bar\beta.\tau_a \to \tau_b, \quad \mathrm{Gen}_\Gamma(e) = (\tau, \eta, C) \Longrightarrow \mathrm{Gen}_\Gamma(\mathbf{perform}\ op\ e) = (\theta\tau_b, \{op\} \cup \eta, C \wedge (\tau = \theta\tau_a)).$$

a, b, η, θ は使用時に新たに選ぶ。

図 7. let の制約生成

$$\mathrm{Gen}_\Gamma(v) = (\tau_1, \varnothing, C_1), \qquad A = \mathrm{FV}(\tau_1, C_1) \setminus \mathrm{FV}(\Gamma),$$
$$\sigma = \forall A.C_1 \Rightarrow \tau_1, \qquad R_1 = \exists A.C_1,$$
$$\mathrm{Gen}_{\Gamma, x:\sigma}(e_2) = (\tau_2, \epsilon_2, C_2).$$

$$\mathrm{Gen}_\Gamma(\mathbf{let}\ x = v\ \mathbf{in}\ e_2) = (\tau_2, \epsilon_2, R_1 \wedge C_2),$$

$$\mathrm{Gen}_\Gamma(e_1) = (\tau_1, \epsilon_1, C_1), \quad \mathrm{Gen}_{\Gamma, x:\tau_1}(e_2) = (\tau_2, \epsilon_2, C_2) \Longrightarrow \mathrm{Gen}_\Gamma(\mathbf{let}\ x = e_1\ \mathbf{in}\ e_2)$$
$$= (\tau_2, \epsilon_1 \cup \epsilon_2, C_1 \wedge C_2), \quad \neg\,\mathrm{Value}(e_1).$$

$$\rho \models R_1 \quad \Longleftrightarrow \quad \exists \rho'.\ \rho' \models C_1 \ \wedge\ \rho'|_{\overline{A}} = \rho|_{\overline{A}}, \qquad \mathrm{FV}(R_1) \subseteq \mathrm{FV}(\Gamma).$$

handler の制約生成

図 8. handler の制約生成

$$\Sigma(op) = \forall \overline{\beta :: \kappa}.\tau_a \to \tau_b, \qquad a, \delta, \chi \text{ fresh},$$
$$\mathrm{Gen}_\Gamma(e) = (\tau_{in}, \epsilon_{in}, C_{in}),$$
$$\mathrm{Gen}_{\Gamma, r:\tau_{in}}(e_r) = (\tau_r, \epsilon_r, C_r),$$
$$\mathrm{Gen}_{\Gamma, x:\tau_a, k:\tau_b \xrightarrow{\chi} a}(e_o) = (\tau_o, \epsilon_o, C_o).$$

$U_T, U_E =$ 操作節の生成中に導入した新しい型・効果変数.

$$\begin{aligned}&\mathrm{Gen}_\Gamma(\mathbf{handle}\ e\ \mathbf{with}\ h) = (a, \chi, C),\\ &\qquad C = C_{in} \wedge C_r \wedge (\tau_r = a)\\ &\qquad\quad \wedge H_X(\bar\beta; U_T, U_E; C_o \wedge \tau_o = a \wedge \epsilon_o \subseteq \chi)\\ &\qquad\quad \wedge \mathrm{Split}(op, \epsilon_{in}, \delta) \wedge \delta \subseteq \chi \wedge \epsilon_r \subseteq \chi.\end{aligned}$$

$\bar\beta$ は剛性変数; $\qquad \bar\beta \cap \mathrm{FV}(\Gamma, \tau_{in}, a, \delta, \chi, \epsilon_{in}, \epsilon_r) = \varnothing.$

$$\rho \models_X H_X(\bar\beta; U; D) \quad \Longleftrightarrow \quad \forall \bar b\, \exists \bar u.\ \rho[\bar\beta \mapsto \bar b, U \mapsto \bar u] \models_X D.$$

H_X は handler にだけ必要な操作であり，任意の制約に対する全称演算を X に要求しない.

$$\mathrm{ClauseScope}(C, \Gamma, \tau_{in}, a, \epsilon_{in}, \epsilon_r, \delta, \chi, U_T, U_E).$$

図 9. *ping* **の例**

$$\Sigma(ping) = \forall\beta.\beta \to \beta, \qquad f : \forall\alpha.\ Eq\,\alpha \Rightarrow \alpha \xrightarrow{\varnothing} \alpha,$$
$$g = \lambda x.\mathbf{perform}\ ping\,(f\,x), \qquad g : \forall\alpha.\ Eq\,\alpha \Rightarrow \alpha \xrightarrow{\{ping\}} \alpha,$$
$$e = \mathbf{handle}\ (g\,y)\ \mathbf{with}\ \{\mathbf{return}\ r \mapsto r;\ ping(x; k) \mapsto k\,x\},$$
$$\mathrm{Split}(ping, \{ping\}, \delta) \Longrightarrow \delta = \varnothing, \qquad \epsilon_r = \varnothing, \qquad \epsilon_o = \chi,$$
$$C_e = Eq\,\alpha \wedge (\varnothing \subseteq \chi) \wedge (\chi \subseteq \chi), \qquad \chi = \varnothing \text{ は一つの解.}$$

1 HM(X) の制約体系

ρ は型変数と効果変数への評価であり，$\rho \models_X C$ は $C \in \mathcal{C}_X$ の充足を表す．X は述語の集合だけではなく

$$X = (\mathcal{C}_X, \models_X, \top, \wedge, \exists\alpha, \exists\mu, =, \subseteq, \notin, H_X)$$

という制約体系である．型等式・効果制約・H_X は EHM(X) が必要とする演算である．基本演算の意味は次の表で固定する．

制約	充足条件
$\top$	常に真
$C \wedge D$	$\rho \models_X C$ かつ $\rho \models_X D$
$\exists\alpha.C$	$\rho[\alpha \mapsto t] \models_X C$ となる t が存在
$\exists\mu.C$	$\rho[\mu \mapsto E] \models_X C$ となる E が存在
$\tau = \tau'$	$I_\rho(\tau) = I_\rho(\tau')$
$\epsilon \subseteq \eta$	$I_\rho(\epsilon) \subseteq I_\rho(\eta)$
$op \notin \epsilon$	$op \notin I_\rho(\epsilon)$

含意は $C \models_X D$，同値は $C \equiv_X D \iff (C \models_X D \wedge D \models_X C)$ とする．これらの意味から結合と存在射影の cylindric laws が従う．制約の自由変数については，そこ以外で等しい二つの評価が制約の真偽を変えないことを要求する．一般化と操作節のため，さらに $\mathrm{FV}(\exists U.C) \subseteq \mathrm{FV}(C) \setminus U$ と $\mathrm{FV}(H_X(B;U;C)) \subseteq \mathrm{FV}(C) \setminus (B \cup U)$ を要求する．基本制約の自由変数が式の自由変数に含まれることも要求する．Lean では `LawfulFV` と `LawfulGeneratedFV` で表す．スキームの具体化は上の射影と等式で定義するため，X に構文的改名や内部束縛名の列挙を要求しない．

操作宣言 $\Sigma(op) = \forall B_T, B_E.\tau_a \to \tau_b$ は閉じている：$\mathrm{FV}_T(\tau_a, \tau_b) \subseteq B_T$ かつ $\mathrm{FV}_E(\tau_a, \tau_b) \subseteq B_E$．操作節の検査は X の一般的な全称演算を必要としない．新しい剛性名 $\bar\beta$ を取り，外側の C が $\bar\beta$ に依存しないとき，

$$C \models_X \exists U.C_o(\bar\beta) \quad\Longleftrightarrow\quad C \models \forall\bar\beta.\exists U.C_o(\bar\beta).$$

右辺の $\forall$ は操作節の検査を説明するメタ論理の記号である．Lean の `ConstraintSystem` は X 全体を表し，`clauseValid_iff_handlerClause` がこの同値を抽象 X について証明する．

Lean で証明した範囲を次に示す．

対象	結果
$X_{\mathrm{syn}}(P)$	handler を含む生成の完全性・主要性．
抽象 X	閉じた操作宣言，`LawfulFV` と `LawfulGeneratedFV` の下で，handler と多相 let を含む生成の完全性・主要性．主要正規化器があれば，閉じた環境で充足可能な主要計算型スキームも得られる．

変数の具体化は射影と等式で表し，X の内部束縛名を改名しない．自由変数法則 `LawfulGeneratedFV` は構文インスタンスで成立する．

2 補題と生成の健全性

補題 2.1 (効果の除去)**.**

$$\rho \models \mathrm{Split}(op, \epsilon, \delta) \quad\Longleftrightarrow\quad I_\rho(\delta) = I_\rho(\epsilon) \setminus \{op\}.$$

したがって Split は op を除いた残りの効果を表す．

補題 2.2 (局所変数の射影)**.** U が型変数と効果変数の有限集合なら，

$$\rho \models \exists U.C \quad\Longleftrightarrow\quad \exists\rho'.\ \rho' \models C \ \wedge\ \rho'|_{\overline{U}} = \rho|_{\overline{U}}.$$

値 **let** の残余制約 $R = \exists A.C_1$ と，操作節の $\forall\bar\beta.\exists U.C_o$ は同じ射影を使う．後者では操作の型引数 $\bar\beta$ を選んだ後で，その節だけの変数 U を選ぶ．

命題 2.3 (制約生成の健全性)**.**

$$\mathrm{Gen}_\Gamma(e) = (\tau, \epsilon, C) \quad \Longrightarrow \quad C; \Gamma \vdash e : \tau \mid \epsilon.$$

実行可能な生成器が返す結果にもこの命題を適用できる．抽象 X 上の規則については，次の *Lean* 定理を使う：

`Generic.generateFresh_sound`

命題 2.4 (制約簡約による解の保存)**.** $\mathrm{Sol}(C) = \{\rho \mid \rho \models C\}$，$\mathrm{Assume}(A) = \bigwedge_{p \in A} p$ とする．

$$\mathrm{simplify}(C) = \mathrm{some}(R) \Longrightarrow \mathrm{Sol}(R) = \mathrm{Sol}(C),$$
$$\mathrm{Sol}(\mathrm{Assume}(A) \wedge \mathrm{resolve}_X(A, C)) = \mathrm{Sol}(\mathrm{Assume}(A) \wedge C).$$

3 完全性と制約付き型の主要性

この節の定理は，閉じた操作宣言と上記の自由変数法則を満たす抽象 X について Lean で証明した．制約付き型を $Q = (C, \tau, \epsilon)$ と書く．$Q_g \succeq_\Gamma Q_d$ は，Q_d の各解に対して，環境 Γ の自由変数の値を変えずに Q_g の解を選べ，型と効果の評価が一致することを表す：

$$\begin{aligned} Q_g \succeq_\Gamma Q_d \quad \Longleftrightarrow \quad \forall \rho_d \models C_d.\ \exists \rho_g \models C_g.\quad & \rho_g|_{\mathrm{FV}(\Gamma)} = \rho_d|_{\mathrm{FV}(\Gamma)} \\ & \wedge I_{\rho_g}(\tau_g) = I_{\rho_d}(\tau_d) \wedge I_{\rho_g}(\epsilon_g) = I_{\rho_d}(\epsilon_d). \end{aligned}$$

定理 3.1 (生成の完全性と主要性)**.** $C_0; \Gamma \vdash e : \tau_0 \mid \epsilon_0$ が導出できるとする．実行可能な生成器は e に対して $Q_g = (C_g, \tau_g, \epsilon_g)$ を返し，

$$C_g; \Gamma \vdash e : \tau_g \mid \epsilon_g, \qquad Q_g \succeq_\Gamma (C, \tau, \epsilon)$$

が，e の任意の宣言的導出 $C; \Gamma \vdash e : \tau \mid \epsilon$ について成り立つ．e には値 **let** と *handler* を含めてよい．

定理 3.2 (推論結果の主要性)**.** 上の定理と同じ前提で，追加の仮定 A を渡した推論器は $Q = (C_q, \tau_q, \epsilon_q)$ を返す．この結果自身に型がつき，任意の $C; \Gamma \vdash e : \tau \mid \epsilon$ について

$$Q \succeq_\Gamma \big(\mathrm{Assume}(A) \wedge C, \tau, \epsilon\big).$$

右辺の制約に解があれば C_q にも解がある．

4 正規化と主要計算型スキーム

ここからは $\mathrm{FV}(\Gamma) = \varnothing$ とする．また X に，充足可能な制約から**主要正規形**を返す正規化器 N_X を仮定する．解決済み制約には $\top$ と同値なものが含まれるとし，$\mathrm{Solved}(R)$ なら $\exists \rho. \rho \models_X R$ とする．これは抽象正規化器への仮定であり，特定の Rust solver に対する証明ではない．Lean では `ConstraintSystem.PrincipalConstraintProperty` と `ConstraintSystem.PrincipalNormalizer` がこの条件を表す．既存の主要計算型スキーム定理は構文的インスタンスの正規化器に適用する．

推論結果 $Q = (C_q, \tau_q, \epsilon_q)$ に新しい出力名 a_o, μ_o を付けて，型と効果も正規化の入力に含める：

$$D_Q = C_q \wedge (a_o = \tau_q) \wedge (\mu_o \equiv \epsilon_q), \qquad N_X(D_Q) = (R, \theta_T, \theta_E).$$

正規形 (R, θ) が表す評価を $\mathrm{Rep}(R, \theta) = \{\theta^* \omega \mid \omega \models R\}$ とする．$\theta^* \omega$ は，型・効果の代入 θ を評価 ω に作用させた結果である．F_D を，その値を変えると D の真偽が変わり得る型・効果変数の集合とする．正規形の一般性は

$$N \succeq_D N' \quad \Longleftrightarrow \quad \mathrm{Rep}(N')|_{F_D} \subseteq \mathrm{Rep}(N)|_{F_D}$$

で比べる．N が主要正規形であるとは，残余制約が解決済みで，$\mathrm{Rep}(N) \subseteq \mathrm{Sol}(D)$ を満たし，同じ条件を満たす任意の N' に対して $N \succeq_D N'$ となることをいう．正規化器は成功時にこの形を返すと仮定する．

定理 4.1 (正規化後の主要性)**.** $C_0;\Gamma \vdash e : \tau_0 \mid \epsilon_0$ が導出でき，$\mathrm{Assume}(A) \wedge C_0$ が充足可能なら，推論と D_Q の正規化は成功する．結果

$$Q_N = \big(R, \theta_T(a_o), \theta_E(\mu_o)\big)$$

は任意の宣言的導出に対して

$$Q_N \succeq_\Gamma \big(\mathrm{Assume}(A) \wedge C, \tau, \epsilon\big) \qquad (C;\Gamma \vdash e : \tau \mid \epsilon)$$

を満たす．$N_X(D_Q)$ 自身も D_Q の主要正規形である．

計算型スキームとその具体例を次のように定める．B は型変数と効果変数の組である．

$$B = \mathrm{FV}(C,\tau,\epsilon) \setminus \mathrm{FV}(\Gamma), \qquad \mathrm{Gen}^{\mathrm{c}}_\Gamma(C,\tau,\epsilon) = \forall B.\ C \Rightarrow (\tau!\epsilon),$$
$$\mathrm{Inst}_\rho(\forall B.C \Rightarrow (\tau!\epsilon)) = \{(I_\omega(\tau), I_\omega(\epsilon)) \mid \omega \models C,\ \omega|_{\overline{B}} = \rho|_{\overline{B}}\},$$
$$\sigma_g \succeq \sigma_d \iff \forall \rho.\ \mathrm{Inst}_\rho(\sigma_d) \subseteq \mathrm{Inst}_\rho(\sigma_g).$$

$\Gamma \vdash_c e : \sigma$ は次の二規則で定める．ここでは Γ が閉じているため，制約の充足可能性だけで一般化の前提を表せる．

$$\frac{C;\Gamma \vdash e : \tau \mid \epsilon \qquad \mathrm{Sol}(C) \neq \varnothing}{\Gamma \vdash_c e : \mathrm{Gen}^{\mathrm{c}}_\Gamma(C,\tau,\epsilon)} \qquad \frac{\Gamma \vdash_c e : \sigma_g \qquad \sigma_g \succeq \sigma_d \qquad \exists \rho.\ \mathrm{Inst}_\rho(\sigma_d) \neq \varnothing}{\Gamma \vdash_c e : \sigma_d}.$$

定理 4.2 (主要計算型スキーム)**.** $\Gamma \vdash_c e : \sigma$ とする．追加仮定なし $(A = \varnothing)$ で推論し，D_Q を正規化して得た $Q_N = (R, \tau_N, \epsilon_N)$ を一般化すると，

$$\sigma_o = \mathrm{Gen}^{\mathrm{c}}_\Gamma(R, \tau_N, \epsilon_N)$$

は閉じた，充足可能な計算型スキームであり，

$$\Gamma \vdash_c e : \sigma_o, \qquad \forall \sigma'.\ \Gamma \vdash_c e : \sigma' \Longrightarrow \sigma_o \succeq \sigma'.$$

すなわち，推論された σ_o 自身に型がつき，同じ項に付けられる全スキームより一般的である．

この結論は handler を含む e について Lean で証明した．

5 逐次的な制約簡約と一般化

値 **let** の右辺を推論した直後に制約を簡約する．簡約器 S_X は正規形を制約として書き戻し，次を満たす：

$$\forall C, \rho.\quad \rho \models_X S_X(C) \iff \rho \models_X C. \tag{Eq}$$

特に $C \models_X \alpha = \mathit{Int}$ なら $S_X(C) \models_X \alpha = \mathit{Int}$ である．したがって環境の $y : \alpha$ に関する等式は，簡約で消えても残余制約に保持される．恒等簡約 $S_X(C) = C$ もこの条件を満たす．具体的な solver が等式を除去するときは，その代入を残余制約に書き戻して (Eq) を満たす必要がある．構文制約の具体例では，Lean の `simplifyTotal` がこの等価性を満たすため，Eq・Show を含むクラス制約体系にも適用できる．

逐次生成器 W_{S_X} は値 **let** で次の順序を使う：

$$W_{S_X}(\Gamma, e_1) = (C_1, \tau_1, \varnothing), \qquad C_1' = S_X(C_1),$$
$$B_1 = \mathrm{FV}(C_1,\tau_1) \setminus \mathrm{FV}(\Gamma), \qquad \sigma_1 = \forall B_1.\ C_1' \Rightarrow \tau_1,$$
$$W_{S_X}(\Gamma, x : \sigma_1, e_2) = (C_2, \tau_2, \epsilon_2).$$

実装では一般化する変数の集合を元の生成器から受け継ぎ，束縛変数の条件だけを C_1' に置き換える．handler の操作節では同じ局所変数の範囲を使う．

定理 5.1 (逐次推論の健全性・完全性・主要性)**.** 上の (Eq)，操作宣言の閉性，自由変数法則の下で，$C_0;\Gamma \vdash e : \tau_0 \mid \epsilon_0$ なら $W_{S_X}(\Gamma, e) = Q_s = (C_s, \tau_s, \epsilon_s)$ が成功し，

$$C_s;\Gamma \vdash e : \tau_s \mid \epsilon_s, \qquad \forall C, \tau, \epsilon.\quad C;\Gamma \vdash e : \tau \mid \epsilon \Longrightarrow Q_s \succeq_\Gamma (C, \tau, \epsilon).$$

また，逐次生成器と元の生成器は成功・失敗が一致する．

定理 5.2 (逐次結果から得る主要計算型スキーム)**.** さらに X の主要正規化器 N_X があり，Γ が閉じ，C_0 が充足可能なら，Q_s の型・効果を出力名に結んだ制約 D_{Q_s} に N_X を適用して得る計算型スキームは，同じ項に付けられるすべての計算型スキームを包含する.

S_X は途中の**等価な制約簡約**，N_X は最後の**主要正規形の計算**である．逐次定理は S_X の等価性を抽象的に仮定しており，Rust の具体的 solver に対する等価性証明ではない.

解釈上の注意. Split$(ping, \{ping\}, \delta)$ なら $\delta = \varnothing$ だが，handler の結果効果 χ は別の変数である．$\chi = \varnothing$ は解の一つであり，包含制約だけから一意には決まらない.

Lean との対応

本文の結果	Lean の定理
制約体系と剛性操作節	`ConstraintSystem.clauseValid_iff_handlerClause`
効果の除去・局所変数の射影	`Constraint.split_iff`, `Constraint.project_holds_iff`, `Generic.Scheme.generalize_instance_iff_residual`
生成と簡約の健全性	`Generic.generateFresh_sound`, `Generate.sound`, `simplify_equiv`, `resolveX_equiv`
抽象 X の核となる式の完全性	`Generic.Typing.core_generate_complete`
抽象 X の実行可能な変数参照の主要性	`Generic.generateFresh_var_principal`, `Generic.Scheme.instantiateProjected_groundInstance`
基本制約の自由変数法則と生成器の番号の単調性	`syntaxSystem_lawfulGeneratedFV`, `Generic.generate_next_mono`, `Generic.generateFresh_support_below`
抽象 X の式構成段階	`Generic.generate_var_coversAt`, `Generic.Output.CoversAt.lam/app/letMono/letVal/perform`
抽象 X の handler を含む完全性・主要性	`Generic.Typing.generate_complete`, `Generic.Typing.generateFresh_principal`
抽象 X の handler を含む主要スキーム	`Generic.Typing.infer_principal_scheme`
逐次生成と主要計算型スキーム	`Generic.generateSequentialFresh_correct`, `Generic.Typing.generateSequentialFresh_principal`, `Generic.Typing.inferSequential_principal_scheme`, `Generic.Typing.classSequentialFresh_principal`
handler のスコープに使う補題	`Generic.generate_effect_fresh`, `Generic.generated_handlerScope_of_runs`, `Generic.generated_clauseScope_of_bounds`
古い自由変数の出所	`Generic.generate_variable_old_support`, `Generic.Output.return_old_below_body`, `Generic.Output.handler_old_support`, `Generic.generate_support_origin`
操作節の量化された証人	`Generic.Output.CoversAt.handlerClause_of_aligned`
抽象 X の値 let・変数・操作節の意味的補題	`Generic.GeneratedCoversAcross.letVal`, `Generic.Context.variable_complete_projected`, `Generic.GeneratedCovers.handlerClause`
抽象 X の正規化後の主要性（生成結果の主要性を前提）	`Generic.ComputationSchemeTyping.infer_principal_scheme_exists_of_mono`
生成の完全性と主要性	`Typing.generate_complete`, `Typing.generateFresh_principal`
推論結果の主要性	`Typing.inferQualifiedTotal_principal`, `Typing.inferQualifiedTotal_realizable`
正規化後の主要性	`Typing.inferQualifiedTotal_normalized_exists`
主要計算型スキーム	`ComputationSchemeTyping.infer_principal_scheme`

付録 A　操作節の局所変数の例

$$\Sigma(ping) = \forall\beta.\beta \to \beta, \qquad id : \forall\gamma.\gamma \xrightarrow{\varnothing} \gamma,$$
$$H_{id} = \lambda y.\mathbf{handle}\ (\mathbf{perform}\ ping\ y)\ \mathbf{with}\ \{\mathbf{return}\ r \mapsto r;\ ping(x;k) \mapsto k\,(id\ x)\}.$$

操作節で id を具体化するときに生成する u は，各 β に依存してよい．主要な制約は次の形になる．

$$\forall\beta.\exists u.\ (u = \beta).$$

$u := \beta$ を選べるので，この部分は充足可能である．ここで射影するのは操作節の生成中に導入した名前だけである．handler の結果型 a や操作の剛性変数 β は射影しない．

$ping(x;k) \mapsto x$ では節に新しい名前がない．従って $\forall\beta.\exists\varnothing.(\beta = a)$ は $\forall\beta.(\beta = a)$ と同じであり，依然として充足不能である．

付録 B　導出木：$ping(x;k) \mapsto x$ の制約

$$\Sigma(ping) = \forall\alpha.\alpha \to \alpha, \qquad \sigma_{id} = \forall\gamma.\gamma \xrightarrow{\varnothing} \gamma, \qquad T = \theta \xrightarrow{\varnothing} \theta,$$
$$B = \textbf{let } id = \lambda z.z \textbf{ in perform } ping\ id,$$
$$H = \textbf{handle } B \textbf{ with } \{\textbf{return } r \mapsto r;\ ping(x;k) \mapsto x\}.$$

本体の導出． id は σ_{id} から T に具体化し，**perform** でも操作型引数を $\alpha := T$ とする．

$$\dfrac{\dfrac{\dfrac{z:\theta \vdash z:\theta \mid \varnothing}{\vdash \lambda z.z : T \mid \varnothing}\,(\text{Abs}) \qquad \dfrac{id:\sigma_{id} \vdash id : T \mid \varnothing}{id:\sigma_{id} \vdash \textbf{perform } ping\ id : T \mid \{ping\}}\,(\text{Perform } \alpha := T)}{\vdash B : T \mid \{ping\}}\,(\text{LetVal})$$

宣言的な Handle 規則の適用． 本体の前提は上で得た．規則全体には return 節と操作節の前提も要る．

$$\dfrac{C;\varnothing \vdash B : T \mid \{ping\} \qquad C; r:T \vdash r : a \mid \varnothing \qquad D; x:\beta,\ k:\beta \xrightarrow{\chi} a \vdash x : \beta \mid \varnothing}{C;\varnothing \vdash H : a \mid \chi}\,(\text{Handle})$$

第 2 前提は $C \models T = a$．操作節では $x:\beta$ が得られ，Handle 規則は $C \models \forall\beta.(D \wedge \beta = a \wedge \varnothing \subseteq \chi)$ を要求する．さらに Split と効果包含が必要である．従って B の型だけでは結論を得られない．

Handler の制約生成． 操作節では別の剛性変数 β を導入する．a は handler の結果型で，$\beta \notin \mathrm{FV}(a)$ である．

$$\dfrac{\mathrm{Gen}_{\varnothing}(B) = (T, \{ping\}, C_B) \qquad \mathrm{Gen}_{r:T}(r) = (T, \varnothing, \top) \qquad \mathrm{Gen}_{x:\beta,\ k:\beta \xrightarrow{\chi} a}(x) = (\beta, \varnothing, \top)}{\mathrm{Gen}_{\varnothing}(H) = (a, \chi, C_H)}\,(\text{Handle})$$

$$C_H = C_B \wedge (T = a) \wedge \underbrace{H_X(\beta;\varnothing;\beta = a \wedge \varnothing \subseteq \chi)}_{\text{操作節の結果型 } \tau_o = a}$$
$$\wedge\, \mathrm{Split}(ping, \{ping\}, \delta) \wedge \delta \subseteq \chi \wedge \varnothing \subseteq \chi.$$

$$\mathrm{Split}(ping, \{ping\}, \delta) \Longrightarrow \delta = \varnothing, \qquad \boxed{\forall\beta.\beta = a \quad \text{は充足不能}}.$$

$\beta = Int$ と $\beta = Bool$ を選ぶと，同じ固定型 a が Int と $Bool$ の両方に等しくなって矛盾する．$C = \top$ では Handle 規則の全称閉包の前提を証明できない．$C = C_H$ とすれば形式上は規則を適用できるが，C_H を満たす評価は存在しない．本体の導出木では，型等式を解いた後の型 T を使った．

$ping(x;k) \mapsto k\,x$ なら，$x:\beta$ と $k:\beta \xrightarrow{\chi} a$ から節の結果型は a となり，$\forall\beta.\beta = a$ は生成されない．

付録 C　元の System F^{ς} で止まる導出木

元の構文には **let** と暗黙の型適用がないため，前頁の式を次の明示的な核の式に直す．

$$\Sigma(\mathit{ping}) = \forall\beta :: \mathbf{T}.\beta \to \beta, \qquad \Delta = \gamma :: \mathbf{T},$$
$$B^{\star} = \mathbf{perform}\ \mathit{ping}[\gamma \xrightarrow{\varnothing} \gamma]\,(\lambda z : \gamma.z),$$
$$H^{\star} = \mathbf{handle}\ B^{\star}\ \mathbf{with}\ \{\mathbf{return}\ r \mapsto r;\ \mathit{ping}(x;k) \mapsto x\}.$$

本体は導出できる．

$$\dfrac{\dfrac{\dfrac{z : \gamma \in (z : \gamma)}{\Delta \mid z : \gamma \vdash z : \gamma \mid \varnothing}\ \textsc{TVar}}{\Delta \mid \varnothing \vdash \lambda z : \gamma.z : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}\ \textsc{TAbs}}{\Delta \mid \varnothing \vdash B^{\star} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\}}\ \textsc{TPerform}\ \beta := \gamma \xrightarrow{\varnothing} \gamma$$

Handler の導出は操作節で止まる．結果型 $\gamma \xrightarrow{\varnothing} \gamma$，結果効果 $\varnothing$ に具体化すると，THandler の効果条件は $\{\mathit{ping}\} \subseteq \{\mathit{ping}\} \cup \varnothing = \{\mathit{ping}\}$ となり成立する．return 節も TVar から $\Delta \mid r : \gamma \xrightarrow{\varnothing} \gamma \vdash r : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing$ と導ける．残る前提だけを枠で示す．

$$\Gamma_{op} = x : \beta, \quad k : \beta \xrightarrow{\varnothing} (\gamma \xrightarrow{\varnothing} \gamma).$$

$$\dfrac{\Delta \mid B^{\star} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\} \qquad \Delta \mid \varnothing \overset{?}{\vdash} \left\{\begin{array}{l}\mathbf{return}\ r \mapsto r;\\ \mathit{ping}(x;k) \mapsto x\end{array}\right\} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\} \Rightarrow \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}{\Delta \mid \varnothing \overset{?}{\vdash} H^{\star} : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}\ \textsc{THandle}$$

ここで

$$\dfrac{\Delta \mid r : \gamma \xrightarrow{\varnothing} \gamma \vdash r : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing \qquad \boxed{\Delta, \beta :: \mathbf{T} \mid \Gamma_{op} \overset{?}{\vdash} x : \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}}{\Delta \mid \varnothing \overset{?}{\vdash} \left\{\begin{array}{l}\mathbf{return}\ r \mapsto r;\\ \mathit{ping}(x;k) \mapsto x\end{array}\right\} : \gamma \xrightarrow{\varnothing} \gamma \mid \{\mathit{ping}\} \Rightarrow \gamma \xrightarrow{\varnothing} \gamma \mid \varnothing}\ \textsc{THandler}$$

枠は未証明の前提であり，これは完成した導出木ではない．操作節の x に TVar を適用すると，

$$\dfrac{x : \beta \in \Gamma_{op}}{\Delta, \beta :: \mathbf{T} \mid \Gamma_{op} \vdash x : \beta \mid \varnothing}\ \textsc{TVar}$$

x の型は β であって $\gamma \xrightarrow{\varnothing} \gamma$ ではない．TSub で関数型に変えるには

$$\boxed{\Delta, \beta :: \mathbf{T} \vdash \beta <: \gamma \xrightarrow{\varnothing} \gamma}$$

が必要だが，元の部分型規則は型変数に対して反射律 $\beta <: \beta$ しか与えず，外側で固定した関数型には導けない．結果型を別の外側の型 R に変えても $\beta <: R$ が必要になる．効果 $\varnothing \subseteq \epsilon_o$ は障害ではない．したがって $H^{\star}$ には元の System F^{ς} の完成した型付け導出木がない．前頁の $\forall\beta.\beta = a$ は，同じ障害を EHM(X) の等式制約で表したもの．